

This assignment is based on the Telecom dataset on which Apache Hadoop is utilized for data streaming and to visualize how Apache Spark handles real-time processing. The data is real and obtained from the statistics bureau Communications Authority of Kenya.

```
In [1]: #imports
import pandas as pd
import pdfplumber
import fitz
import os
import glob
import subprocess
import sys
import urllib.request
import urllib.error
from pathlib import Path
import importlib.util
```

1. The first step is to extract the data from the reports and convert them to CSV.

```
In [3]: # PDF files to extract from
pdf_files = [
    "_Sector Statistics Report Q3 2025-2026.pdf",
    "Sector Statistics Report Q2 2025-2026.pdf",
    "Sector Statistics Report Q1 2025-2026_0.pdf"
]

all_charts = []

for file in pdf_files:
    pdf_path = file

    if os.path.exists(pdf_path):
        print(f"Processing {pdf_path}...")

        # 1. Extract Tables using pdfplumber
        output_folder = "ExtractedTables"
        os.makedirs(output_folder, exist_ok=True)

        with pdfplumber.open(pdf_path) as pdf:
            for page_no, page in enumerate(pdf.pages, start=1):
                tables = page.extract_tables()
                if not tables:
                    continue
                for i, table in enumerate(tables):
                    df = pd.DataFrame(table)
                    filename = os.path.join(
                        output_folder,
                        f"{os.path.splitext(file)[0]}_Table_{i+1}_Page_{page_no}.csv"
                    )
                    df.to_csv(filename, index=False, header=False)
```

```

# 2. Extract Charts and Images using fitz (PyMuPDF)
doc = fitz.open(pdf_path)
for i, page in enumerate(doc):
    # Search for labels usually associated with charts
    text_instances = page.search_for("Figure") + page.search_for("Chart")
    if text_instances:
        text = page.get_text()
        lines = text.split('\n')
        for idx, line in enumerate(lines):
            if "Figure" in line or "Chart" in line:
                title = line.strip()
                # Capture surrounding text which might contain chart data
                start_idx = 0 if idx < 2 else idx - 2
                context = " | ".join(lines[start_idx:idx+10]).strip()
                all_charts.append({
                    "Source": pdf_path,
                    "Page": i + 1,
                    "Type": "Chart",
                    "Label/Title": title,
                    "Content": context
                })
        doc.close()
    else:
        print(f"File not found: {pdf_path}")

# Compile charts to CSV dataset
if all_charts:
    charts_df = pd.DataFrame(all_charts)
    charts_df.to_csv("charts_data.csv", index=False)
    print(f"Charts saved to charts_data.csv ({len(all_charts)} charts compiled)")

if not all_charts:
    print("No chart was extracted.")

```

Processing _Sector Statistics Report Q3 2025-2026.pdf...
 Processing Sector Statistics Report Q2 2025-2026.pdf...
 Processing Sector Statistics Report Q1 2025-2026_0.pdf...
 Charts saved to charts_data.csv (108 charts compiled)

2. Clean the data and prepare for hadoop processing.

```

In [4]: folder = "ExtractedTables" #folder containing the extracted tables
csv_files = glob.glob(os.path.join(folder, "*.csv"))
print(f"Found {len(csv_files)} tables")

def clean_table(df):
    # Remove completely empty rows
    df = df.dropna(how="all")
    # Remove completely empty columns
    df = df.dropna(axis=1, how="all")
    # Strip spaces
    df = df.apply(lambda col: col.astype(str).str.strip())
    # Replace "None" strings

```

```

df.replace(
    ["None", "nan", ""],
    pd.NA,
    inplace=True
)
return df

#save the cleaned tables into a new folder
output_folder = "CleanedTables"
os.makedirs(output_folder, exist_ok=True)
for file in csv_files:
    df = pd.read_csv(file, header=None)
    df = clean_table(df)
    output = os.path.join(
        output_folder,
        os.path.basename(file)
    )
    df.to_csv(output, index=False, header=False)

#merge the dataset
# master_list = []
# for file in glob.glob("CleanedTables/*.csv"):
#     df = pd.read_csv(file)
#     if not df.empty:
#         df["Table_Name"] = os.path.basename(file)
#         df.fillna(0, inplace=True)
#         # Drop all-NA columns to avoid FutureWarning during concat
#         df = df.dropna(how="all", axis=1)
#         if not df.empty:
#             master_list.append(df)
#
# if master_list:
#     master = pd.concat(master_list, ignore_index=True)
#     master.to_csv(
#         "telecom_master.csv",
#         index=False
#     )
# else:
#     print("No data to merge.")

```

Found 110 tables

3. Set up Hadoop and HDFS environment.

```

In [5]: def check_hadoop_web_ui(url="http://localhost:9870"):
        """
        Check whether Hadoop NameNode Web UI is reachable from Windows.
        """
        try:
            with urllib.request.urlopen(url, timeout=5) as response:
                return response.status == 200
        except urllib.error.URLError as e:
            print(f"Could not reach Hadoop Web UI: {e}")
            return False

```

```

def run_wsl_command(command):
    """
    Run a Linux command inside Ubuntu WSL from a Windows Python notebook.
    """
    # Use -d Ubuntu-22.04 to ensure we use the correct distro
    # Use bash -lc to run the command in a login shell
    full_command = ["wsl", "-d", "Ubuntu-22.04", "bash", "-lc", command]
    result = subprocess.run(
        full_command,
        capture_output=True,
        text=True,
        check=False
    )
    if result.stdout.strip():
        print(result.stdout)
    if result.stderr.strip():
        print(result.stderr)
    return result

def windows_path_to_wsl_path(path):
    """
    Convert a Windows path like D:\\folder\\file.csv
    into a WSL path like /mnt/d/folder/file.csv.
    """
    path = Path(path).resolve()
    drive = path.drive.replace(":", "").lower()
    parts = path.parts[1:]
    return "/mnt/" + drive + "/" + "/".join(parts).replace("\\", "/")

project_root = Path.cwd()
project_root_wsl = windows_path_to_wsl_path(project_root)

print("Windows project root:", project_root)
print("WSL project root:", project_root_wsl)

run_wsl_command("which spark-submit && spark-submit --version")

def check_hadoop_services_wsl():
    """
    Check Hadoop Java services inside WSL.
    """
    result = run_wsl_command("jps")

    services = [
        "NameNode",
        "DataNode",
        "SecondaryNameNode",
        "ResourceManager",
        "NodeManager"
    ]

    running = [
        service
        for service in services

```

```

        if service in result.stdout
    ]

    return running

if check_hadoop_web_ui():
    print("✅ Hadoop NameNode Web UI is reachable at http://localhost:9870")
else:
    print("❌ Hadoop NameNode Web UI is not reachable from Windows.")

running_services = check_hadoop_services_wsl()
print(f"Running Hadoop services in WSL: {running_services}")

if "NameNode" in running_services and "DataNode" in running_services:
    print("\n✅ HDFS is running in WSL and ready to use.")
else:
    print("\n⚠️ HDFS services were not detected through WSL.")
    print("If the web UI works, Hadoop may still be running, but 'jps' may not be a

```

Windows project root: D:\Msc-AI\DataMining n BigData\Module3 assignment

WSL project root: /mnt/d/Msc-AI/DataMining n BigData/Module3 assignment

✅ Hadoop NameNode Web UI is reachable at http://localhost:9870

1840 SparkSubmit

673 NameNode

15204 Jps

794 DataNode

5628 SparkSubmit

1292 ResourceManager

1006 SecondaryNameNode

1422 NodeManager

Running Hadoop services in WSL: ['NameNode', 'DataNode', 'SecondaryNameNode', 'ResourceManager', 'NodeManager']

✅ HDFS is running in WSL and ready to use.

4. Stream the data from HDFS to Spark.

```

In [ ]: print("🔗 Connecting to WSL Spark cluster...")

# Create a Spark session
streaming_script = r'''
#!/usr/bin/env python3

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
import os
import time

# Set environment (for safety)
os.environ['SPARK_HOME'] = '/home/locha/spark-4.1.2-bin-hadoop3'
os.environ['HADOOP_HOME'] = '/home/locha/hadoop-3.5.0'
os.environ['JAVA_HOME'] = '/usr/lib/jvm/java-17-openjdk-amd64'

```

```

# Create Spark session
spark = SparkSession.builder \
    .appName("TelecomHDFSStreaming") \
    .master("local[*]") \
    .config("spark.sql.streaming.schemaInference", "true") \
    .config("spark.sql.shuffle.partitions", "4") \
    .config("spark.driver.memory", "2g") \
    .config("spark.executor.memory", "2g") \
    .config("spark.hadoop.fs.defaultFS", "hdfs://localhost:9000") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

print(f"✅ Spark {spark.version} ready!")
print(f"🌐 Spark UI: http://localhost:4040")

# Path to HDFS telecom data
hdfs_path = "hdfs://localhost:9000/telecom/"
print(f"📡 Streaming from: {hdfs_path}")

try:
    # Check if HDFS has data
    hdfs_files = spark.sparkContext._jvm.org.apache.hadoop.fs.FileSystem.get(
        spark.sparkContext._jsc.hadoopConfiguration()
    ).listStatus(
        spark.sparkContext._jvm.org.apache.hadoop.fs.Path(hdfs_path)
    )

    file_count = len(hdfs_files)
    print(f"📁 Found {file_count} files in HDFS /telecom")

    if file_count == 0:
        print("⚠️ No files found! Please upload data to HDFS.")
        print("    hdfs dfs -put /home/hadoop-projects/CleanedTables/*.csv /telecom/")
        spark.stop()
        exit(0)

except Exception as e:
    print(f"⚠️ Could not access HDFS: {e}")
    print("    Make sure Hadoop services are running:")
    print("    cd /home/locha/hadoop-3.5.0/sbin && ./start-dfs.sh")

# Read streaming data from HDFS
stream_df = spark.readStream \
    .option("header", "false") \
    .option("maxFilesPerTrigger", 1) \
    .option("recursiveFileLookup", "true") \
    .csv(hdfs_path)

print(f"✅ Streaming DataFrame created!")

# Add processing metadata
processed_stream = stream_df \
    .withColumn("processing_time", current_timestamp()) \
    .withColumn("source_file", input_file_name()) \

```

```

        .withColumn("batch_id", monotonically_increasing_id())

# Query 1: Show live data stream
query1 = processed_stream.writeStream \
    .outputMode("append") \
    .format("console") \
    .option("truncate", "false") \
    .option("numRows", 10) \
    .trigger(processingTime="5 seconds") \
    .start()

print(f"✅ Query 1 started: Console Output")

# Query 2: Save to HDFS Parquet for analysis
output_hdfs = "hdfs://localhost:9000/telecom_streaming_analytics/"
query2 = processed_stream.writeStream \
    .outputMode("append") \
    .format("parquet") \
    .option("path", output_hdfs) \
    .option("checkpointLocation", "/tmp/checkpoints") \
    .trigger(processingTime="10 seconds") \
    .start()

print(f"✅ Query 2 started: Saving to {output_hdfs}")

print("""
=====
🚀 STREAMING ACTIVE
=====
Monitoring: http://localhost:4040
HDFS Data: http://localhost:9870/explorer.html#/telecom

Press Ctrl+C to stop streaming...
=====
""")

# Keep streaming running
try:
    spark.streams.awaitAnyTermination()
except KeyboardInterrupt:
    print("\n❏ Stopping streaming...")
    spark.streams.stop()
    print(f"✅ Streaming stopped")
finally:
    spark.stop()
...

# Save the streaming script
streaming_script_path = Path.cwd() / "telecom_streaming.py"
streaming_script_path.write_text(streaming_script, encoding="utf-8")

print(f"✅ Streaming script created at: {streaming_script_path}")

```

5. Visualize the streaming data using Spark UI and HDFS Explorer.

```

In [ ]: visualization_script = '''
#!/usr/bin/env python3

"""
Real-time visualization of telecom streaming data
"""

import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import subprocess
import tempfile
import os
from datetime import datetime
import time

def get_hdfs_data():
    """Get latest streaming data from HDFS"""
    try:
        # Create temp directory
        temp_dir = tempfile.mkdtemp()

        # Get latest parquet files from HDFS
        cmd = f'hdfs dfs -ls hdfs://localhost:9000/telecom_streaming_analytics/ | g
        result = subprocess.run(cmd, shell=True, capture_output=True, text=True)

        if result.stdout:
            files = result.stdout.strip().split('\n')
            data_frames = []
            for f in files:
                if f:
                    # Download file
                    local_path = os.path.join(temp_dir, os.path.basename(f))
                    subprocess.run(f'hdfs dfs -get {f} {local_path}', shell=True, c
                    if os.path.exists(local_path):
                        df = pd.read_parquet(local_path)
                        data_frames.append(df)

            if data_frames:
                return pd.concat(data_frames, ignore_index=True)

        except Exception as e:
            print(f"⚠️ Error reading HDFS: {e}")

        return None

def create_dashboard():
    """Create real-time dashboard"""
    plt.style.use('seaborn-v0_8')

    # Create figure with subplots
    fig, axes = plt.subplots(2, 2, figsize=(15, 10))
    fig.suptitle('Telecom Real-Time Streaming Dashboard', fontsize=16)

    def update(frame):

```



```

"""Update dashboard with new data"""
df = get_hdfs_data()

if df is None or df.empty:
    for ax in axes.flatten():
        ax.clear()
        ax.text(0.5, 0.5, 'Waiting for data...', ha='center', va='center')
    return

# Clear axes
for ax in axes.flatten():
    ax.clear()

# 1. Records over time
if 'processing_time' in df.columns:
    df['processing_time'] = pd.to_datetime(df['processing_time'])
    time_counts = df.groupby(df['processing_time'].dt.floor('min')).size()
    axes[0,0].plot(time_counts.index, time_counts.values, 'b-')
    axes[0,0].set_title('Records per Minute')
    axes[0,0].set_xlabel('Time')
    axes[0,0].set_ylabel('Record Count')
    axes[0,0].tick_params(axis='x', rotation=45)

# 2. Column distribution
axes[0,1].bar(['Columns'], [len(df.columns)])
axes[0,1].set_title('Number of Columns')
axes[0,1].set_ylabel('Count')

# 3. Sample data table
sample = df.head(5)
if not sample.empty:
    axes[1,0].axis('tight')
    axes[1,0].axis('off')
    axes[1,0].table(
        cellText=sample.values,
        colLabels=sample.columns,
        loc='center',
        cellLoc='center'
    )
    axes[1,0].set_title('Latest Records')

# 4. Data statistics
stats = {
    'Total Records': len(df),
    'Columns': len(df.columns),
    'Memory Usage': f"{df.memory_usage().sum() / 1024:.2f} KB",
    'Last Update': datetime.now().strftime('%H:%M:%S')
}
axes[1,1].axis('tight')
axes[1,1].axis('off')
axes[1,1].table(
    cellText=[[k, v] for k, v in stats.items()],
    colLabels=['Metric', 'Value'],
    loc='center'
)
axes[1,1].set_title('Streaming Statistics')

```

```
plt.tight_layout()

ani = FuncAnimation(fig, update, interval=5000)
plt.show()

if __name__ == "__main__":
    print("✎ Starting Telecom Streaming Dashboard...")
    print("    This will update every 5 seconds")
    create_dashboard()
...

visualization_path = Path.cwd() / "telecom_dashboard.py"
visualization_path.write_text(visualization_script, encoding="utf-8")

print(f"✅ Dashboard script created at: {visualization_path}")
```